Esercizio 1

Introduzione

Questa relazione presenta un'analisi comparativa delle prestazioni di due algoritmi di ordinamenti: Merge Sort e Quick Sort. Lo studio si è concentrato sulla valutazione dei tempi di esecuzione di questi algoritmi applicati a tre diverse tipologie di dati: stringhe, numeri interi e decimali.

Il Merge Sort,

funziona bene con dataset grandi ed è più stabile nei tempi grazie alla sua complessità prevedibile di O(nlogn). Tuttavia, con dataset molto grandi, la gestione della memoria, specialmente per campi basati su stringhe, può rallentarlo.

Il Quick Sort,

invece, può essere più veloce in alcuni casi, specialmente se i dati sono già ben suddivisi. Però, nel caso peggiore, le sue prestazioni possono peggiorare fino a $O(n^2)$. Ordinare stringhe (campo1) richiede meno tempo, grazie alla partizione a tre vie, rispetto ai numeri interi (campo2) o decimali (campo3),

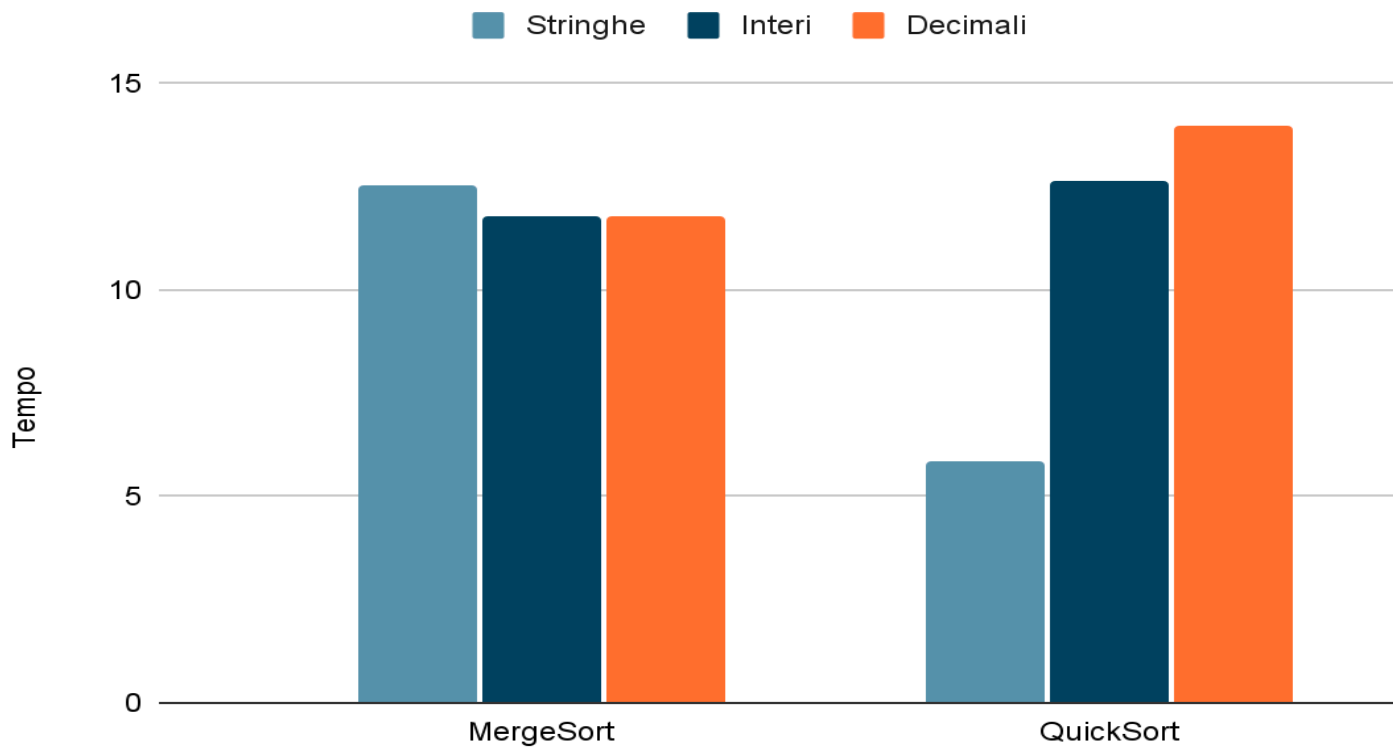
Discussione dei risultati ottenuti

Un punto a favore di Merge Sort è che è stabile: se due valori sono uguali, mantiene l'ordine originale tra di loro, il che può essere utile per certi tipi di dataset.
In generale, entrambi gli algoritmi hanno funzionato bene, ma il Merge Sort è stato più veloce per i numeri interi e decimali.
Il Quick Sort, comunque, è stato il più rapido con le stringhe.
Indipendentemente dall'algoritmo usato, non ci sono stati timeout entro il limite di 10 minuti.

Tabella contenente i tempi di esecuzione (in secondi)

	Stringhe	Interi	Decimali
MergeSort	12.526910	11.806732	11.780020
QuickSort	5.827376	12.640888	13.992397

Grafici



Conclusioni

Sulla base dei risultati ottenuti, possiamo concludere che:

1) Il Quick Sort è l'opzione preferibile per l'ordinamento di stringhe, dove mostra prestazioni nettamente superiori.
2) Il Merge Sort rappresenta la scelta migliore per dati numerici, offrendo prestazioni più efficienti e prevedibili.
3) In scenari dove la stabilità dell'ordinamento è un requisito fondamentale, il Merge Sort dovrebbe essere preferito indipendentemente dal tipo di dato.

La scelta dell'algoritmo ottimale dipenderà quindi principalmente dal tipo di dato da ordinare e dai requisiti specifici dell'applicazione in termini di stabilità e prevedibilità delle prestazioni.